

⚡ JobSpark ### Plataforma de Recrutamento & Seleção com IA Bidirecional *Match inteligente candidato ↔ vaga, transparente para os dois lados.* [![Live](https://img.shields.io/badge/demo-online-success?style=flat-square)](https://jobspark-frontend-231292456749.southamerica-east1.run.app) [![Next.js](https://img.shields.io/badge/Next.js-15-black?style=flat-square&logo=next.js)](https://nextjs.org/) [![Hono](https://img.shields.io/badge/Hono-4-orange?style=flat-square&logo=hono)](https://hono.dev/) [![Bun](https://img.shields.io/badge/Bun-1-fbf0df?style=flat-square&logo=bun)](https://bun.sh/) [![Prisma](https://img.shields.io/badge/Prisma-6-2D3748?style=flat-square&logo=prisma)](https://www.prisma.io/) [![PostgreSQL](https://img.shields.io/badge/PostgreSQL-pgvector-336791?style=flat-square&logo=postgresql)](https://github.com/pgvector/pgvector) [![Cloud Run](https://img.shields.io/badge/Deploy-Cloud%20Run-4285F4?style=flat-square&logo=googlecloud)](https://cloud.google.com/run) **[🌐 Acessar a demo ao vivo](https://jobspark-frontend-231292456749.southamerica-east1.run.app)**

Sobre

JobSpark é uma plataforma SaaS de recrutamento que conecta candidatos e empresas com **IA transparente**. Diferente das soluções tradicionais (onde o candidato fica "cego"), aqui **os dois lados enxergam o match**: o candidato vê seu percentual de aderência a cada vaga *antes* de se candidatar, e a empresa recebe os candidatos já ranqueados por compatibilidade.

A inteligência é dupla: - **Busca semântica** por *embeddings* vetoriais (pgvector) — encontra vagas por significado, não por palavra-chave. - **Análise por LLM** — feedback de currículo e avaliação estruturada de match (skills compatíveis, lacunas, sugestões de desenvolvimento, estimativa salarial).

Funcionalidades

Para o Candidato

- 🔍 **Feed de vagas** com *match score* personalizado por skills
- 💬 **Busca semântica** ("quero trabalhar com IA em startup remota")
- 🎯 **Ranking Top 5** de vagas por similaridade vetorial com o perfil
- 📊 **Análise de candidatura por IA** (skills compatíveis, lacunas, sugestões, estimativa salarial)
- 👤 **Perfil editável** (headline, bio, skills, links, pretensão salarial)
- 📅 **Acompanhamento de candidaturas** com status em tempo real

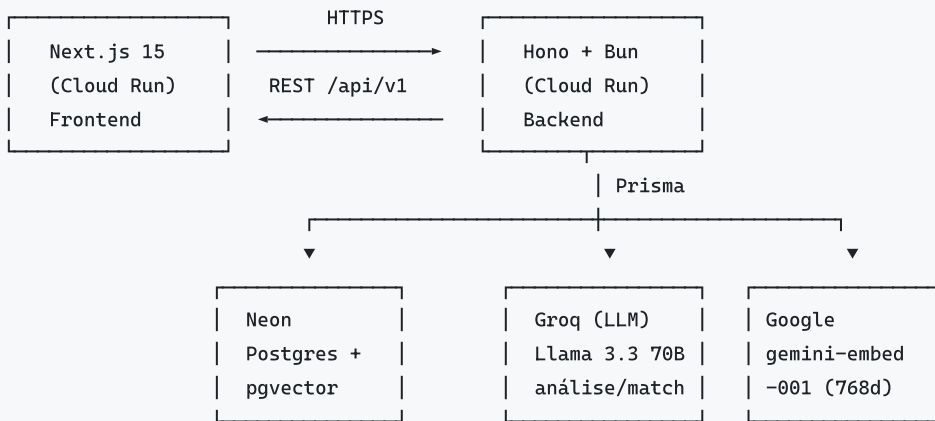
Para a Empresa

- 📄 **CRUD de vagas** com **transparência salarial obrigatória**
- 📁 **Pipeline de candidatos** ranqueados por *match score*
- 🔄 **Atualização de status** da candidatura (triagem → contratação)

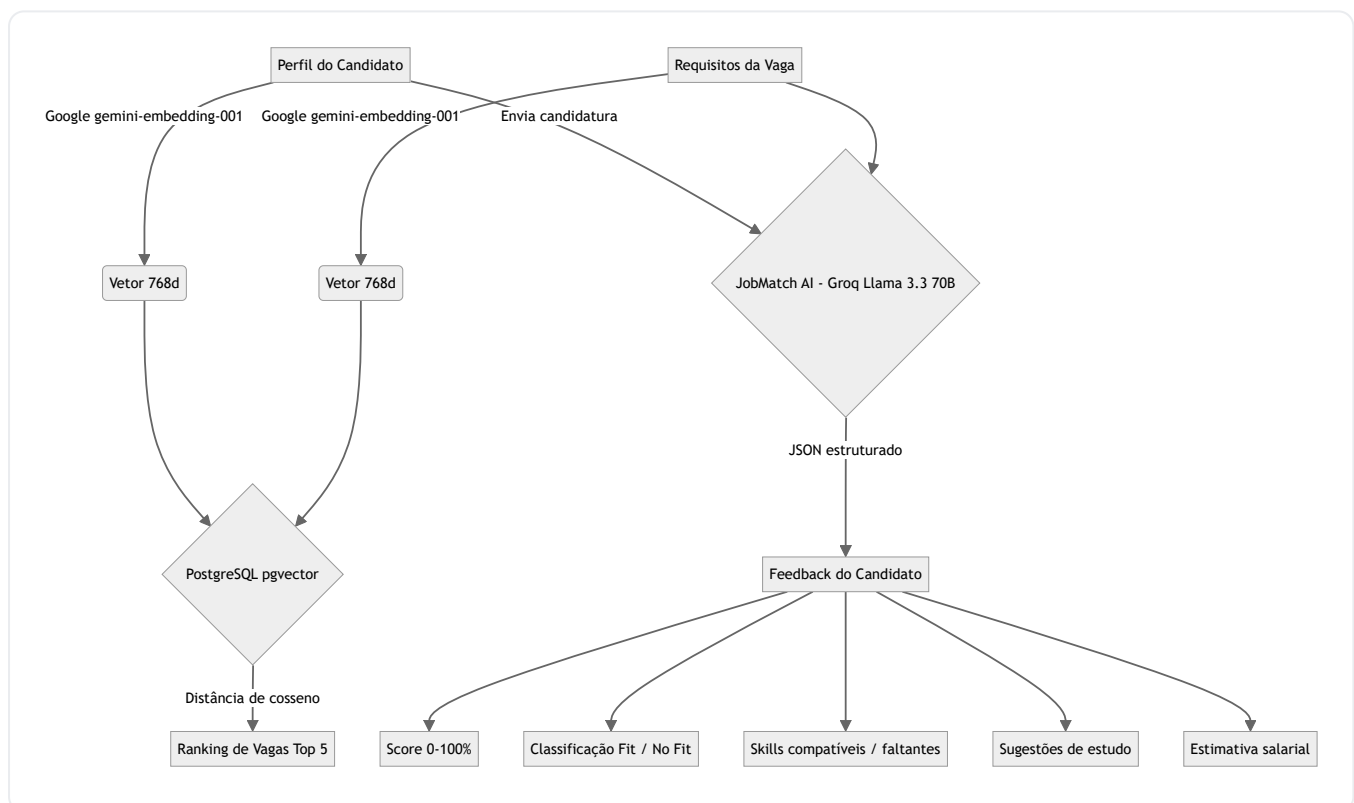
-  **Perfil público** da empresa com vagas ativas



Arquitetura



Fluxo do JobMatch AI



Notas de engenharia: secrets no **Google Secret Manager** (nunca na imagem); embeddings com *fallback* local determinístico quando não há chave; serviços **escalam a zero** no Cloud Run (custo ~nulo em ociosidade).



Camada	Tecnologias
Frontend	Next.js 15 (App Router, standalone), React 19, Tailwind CSS v4, Zustand, React Hook Form, Lucide, Framer Motion
Backend	Hono.js, Bun, Prisma ORM, JWT (hono/jwt), Bun.password (hashing)
Banco	PostgreSQL + pgvector (Neon serverless)
IA / ML	Groq — Llama 3.3 70B (análise/match) · Google gemini-embedding-001 (embeddings 768d)
Infra	Google Cloud Run · Artifact Registry · Cloud Build · Secret Manager · Docker



Começando (ambiente local)

Pré-requisitos

Bun (backend) · Node.js 20+ (frontend) · Docker (Postgres local)

1. Subir o banco local (Postgres + pgvector)

```
cd infra && docker compose up -d
```

2. Backend

```
cd backend
cp .env.example .env          # preencha as chaves (veja abaixo)
bun install
bun run db:migrate             # cria tabelas + extensão vector
bun run db:seed                # popula dados de demonstração
bun dev                        # http://localhost:3002
```

3. Frontend

```
cd frontend
npm install
npm run dev                    # http://localhost:3000
```



Variáveis de Ambiente

Backend (backend/.env)

```
DATABASE_URL=postgresql://user:pass@host:5432/db # Postgres com pgvector
PORT=3002

# IA - Groq (https://console.groq.com/keys)
GROQ_API_KEY=
GROQ_MODEL=llama-3.3-70b-versatile

# Embeddings - Google (https://aistudio.google.com/app/apikey)
GOOGLE_AI_API_KEY=
GOOGLE_EMBED_MODEL=gemini-embedding-001

# Auth - gere com: node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
JWT_SECRET=
BETTER_AUTH_SECRET=
```

💡 Sem GROQ_API_KEY o LLM roda em **modo mock**; sem GOOGLE_AI_API_KEY os embeddings usam o **fallback local determinístico**. A aplicação funciona nos dois casos.

Frontend (frontend/.env.local)

```
NEXT_PUBLIC_API_URL=http://localhost:3002
NEXT_PUBLIC_APP_URL=http://localhost:3000
```



API (REST /api/v1)

Método	Rota	Descrição	Auth
POST	/auth/register	Cadastro (candidato ou empresa)	—
POST	/auth/login	Login (retorna JWT)	—
GET	/auth/me	Usuário autenticado	✅
GET	/jobs	Feed de vagas (+ match score se logado)	opcional
POST	/jobs	Criar vaga	🏢
GET	/jobs/:id	Detalhe da vaga	—
PUT / DELETE	/jobs/:id	Editar / excluir vaga	🏢
GET	/jobs/company	Vagas da empresa logada	🏢
POST	/applications	Candidatar-se (gera análise de IA)	👤
GET	/applications	Minhas candidaturas	👤
GET	/applications/job/:jobId	Candidatos de uma vaga	🏢
PATCH	/applications/:id/status	Atualizar status	🏢
GET / PUT	/candidates/me , /candidates/profile	Perfil do candidato	👤
GET / PUT	/companies/me , /companies/profile	Perfil da empresa	🏢
POST	/ai/search	Busca semântica de vagas	—
POST	/ai/analyze-resume	Feedback de currículo por IA	—
GET	/ai/ranking	Top 5 vagas para o candidato	👤



Estrutura do Projeto

```
jobs-platform/
├── backend/
│   ├── src/
│   │   ├── routes/      # auth · jobs · candidates · companies · applications · ai
│   │   ├── lib/         # prisma · gemini (IA: Groq + embeddings Google) · authUtils
│   │   ├── middleware/  # auth (JWT)
│   │   ├── seed.ts      # dados de demonstração
│   │   └── index.ts     # entry point Hono
│   ├── prisma/          # schema + migrations
│   └── Dockerfile
├── frontend/
│   ├── src/app/         # App Router: (auth) (candidate) (company)
│   ├── src/lib/         # api client · authStore (Zustand)
│   ├── cloudbuild.yaml  # build com a URL da API embutida
│   └── Dockerfile
├── infra/               # docker-compose (Postgres local)
└── docs/                # PRD
```



Deploy (Google Cloud Run + Neon)

Resumo do fluxo usado em produção:

```
# 1. Projeto + APIs
gcloud projects create jobspark-prod-XXXX
gcloud services enable run cloudbuild artifactregistry secretmanager

# 2. Secrets
printf '%s' "$GROQ_KEY" | gcloud secrets create groq-api-key --data-file=-
# ... idem para google-ai-api-key, jwt-secret, better-auth-secret, database-url

# 3. Migrations no Neon (endpoint DIRECT, sem -pooler)
DATABASE_URL="$NEON_DIRECT" bunx prisma migrate deploy

# 4. Backend → Cloud Run
gcloud builds submit --tag <região>-docker.pkg.dev/<proj>/jobspark/backend backend/
gcloud run deploy jobspark-backend --image ... --set-secrets ...

# 5. Frontend → Cloud Run (build com a URL do backend)
gcloud builds submit --config frontend/cloudbuild.yaml \
  --substitutions=_API_URL=<backend-url>,_IMAGE=...
gcloud run deploy jobspark-frontend --image ...
```

Neon usa **direct** para migrations e **pooled + pgbouncer=true** em runtime. O

NEXT_PUBLIC_API_URL é embutido em *build time*, então o frontend é construído **após** o backend.



Demonstração

Perfil	E-mail	Senha
 Empresa	empresa@demo.com	demo1234
 Candidato	candidato@demo.com	demo1234



Roadmap

- [x] Autenticação, CRUD de vagas, candidaturas
- [x] Match por skills + análise de candidatura por IA
- [x] Busca semântica e ranking por embeddings (pgvector)
- [x] Deploy em produção (Cloud Run + Neon)
- [] Upload real de currículo (Cloud Storage / R2)
- [] Pipeline Kanban de candidatos
- [] Notificações em tempo real (SSE/WebSocket)
- [] Testes automatizados (Vitest + Playwright)
- [] Planos de assinatura (Stripe)



Autor

Idarlan Magalhães — Desenvolvedor focado em IA & Visão Computacional [GitHub](#) · [LinkedIn](#)

Construído com ⚡ Next.js, Hono, Bun e IA (Groq + Google).